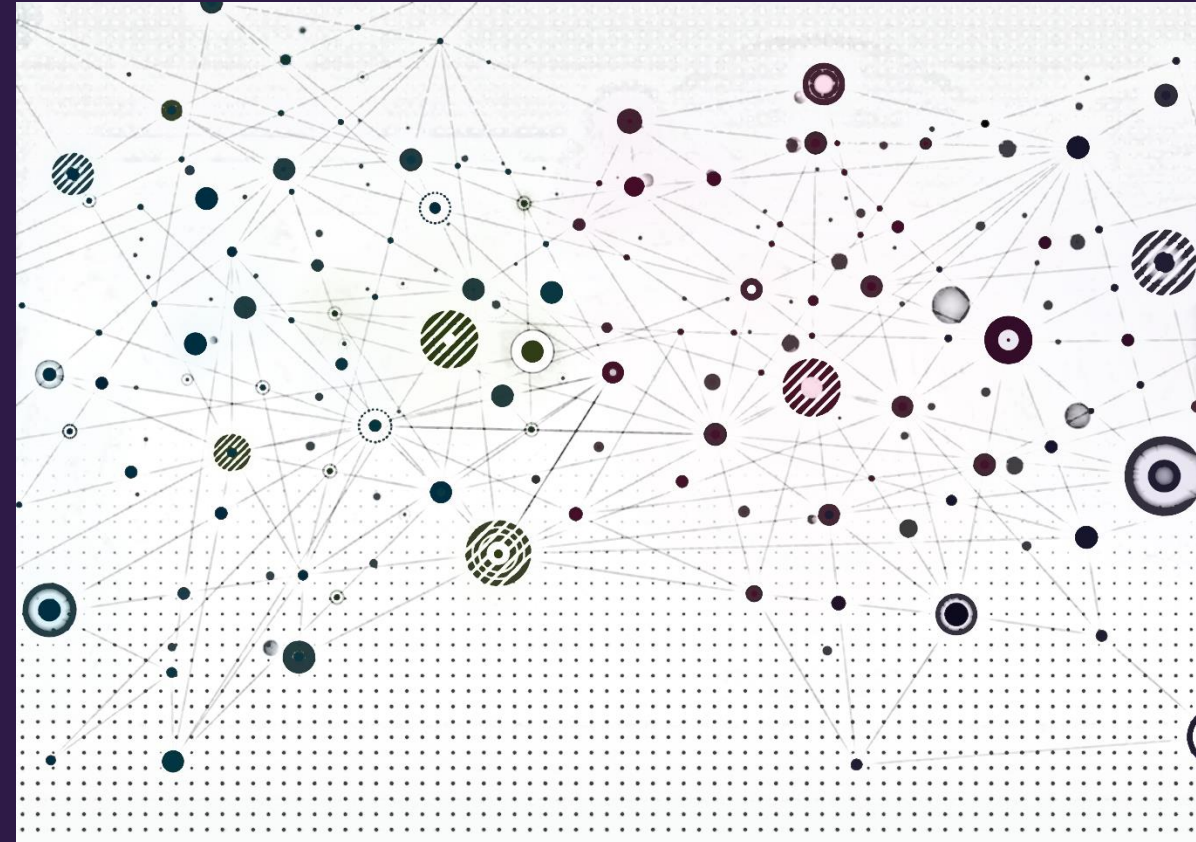


Geometric graphs





Outline

- What is a geometric graph?
- Motivation for geometric graphs
- Constructing a geometric graph
- Geometric symmetry
- Molecular dynamics as a template
- Invariant Geometric GNNs
- Equivariant Geometric GNNs



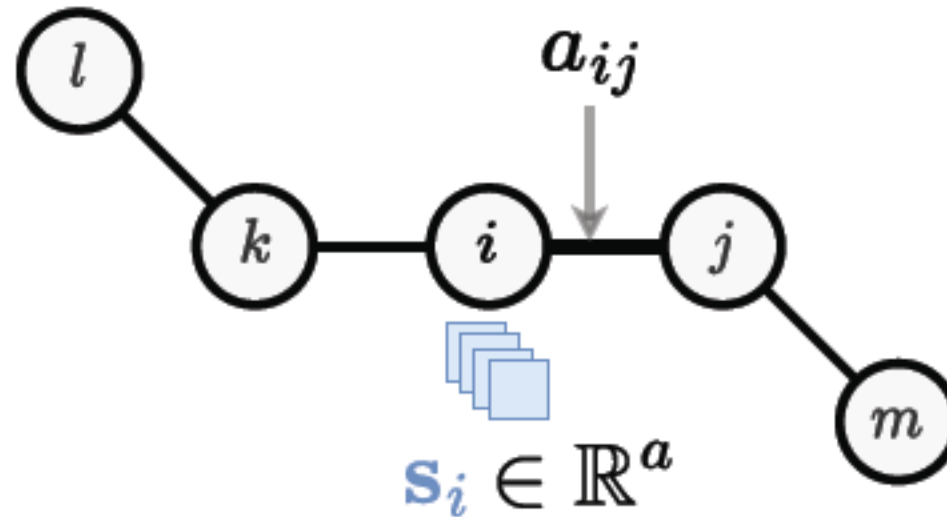
What is a geometric graph?





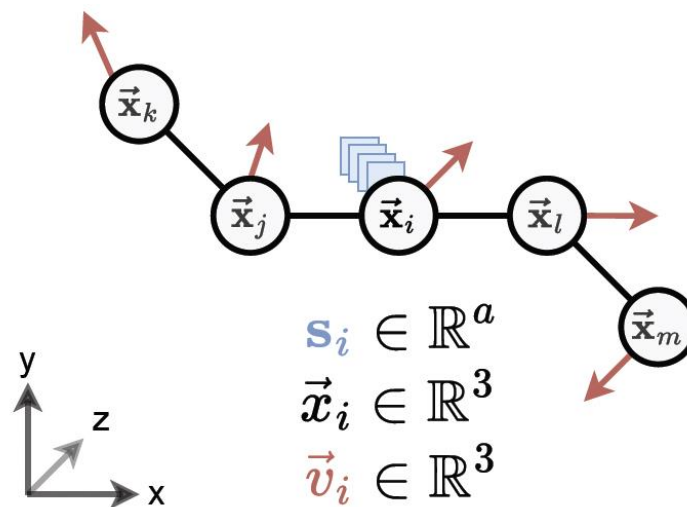
Attributed graph

- An **attributed** graph, $\mathcal{G} = (\mathbf{A}, \mathbf{S})$, is a set, $\mathcal{V}$, of n nodes connected by edges in $\mathcal{E}$. Each node has scalar attributes (features).
 - $\mathbf{A}$ is the $n \times n$ adjacency matrix
 - $\mathbf{S} \in \mathbb{R}^{n \times a}$ is the scalar feature matrix
- This is what we've been working with all semester!



Geometric graph

- A **geometric** graph, $\mathcal{G} = (\mathbf{A}, \mathbf{S}, \vec{\mathbf{v}}, \vec{\mathbf{x}})$, is an attributed graph that is also decorated with geometric attributes:
 - $\vec{\mathbf{x}} \in \mathbb{R}^{n \times d}$ are the node coordinates in a d -dimension Euclidean space
 - $\vec{\mathbf{v}} \in \mathbb{R}^{n \times b \times d}$ optional vector features (e.g., forces)
 - Typically, $d = 3$ corresponding to a 3D coordinate system





Motivation for geometric graphs





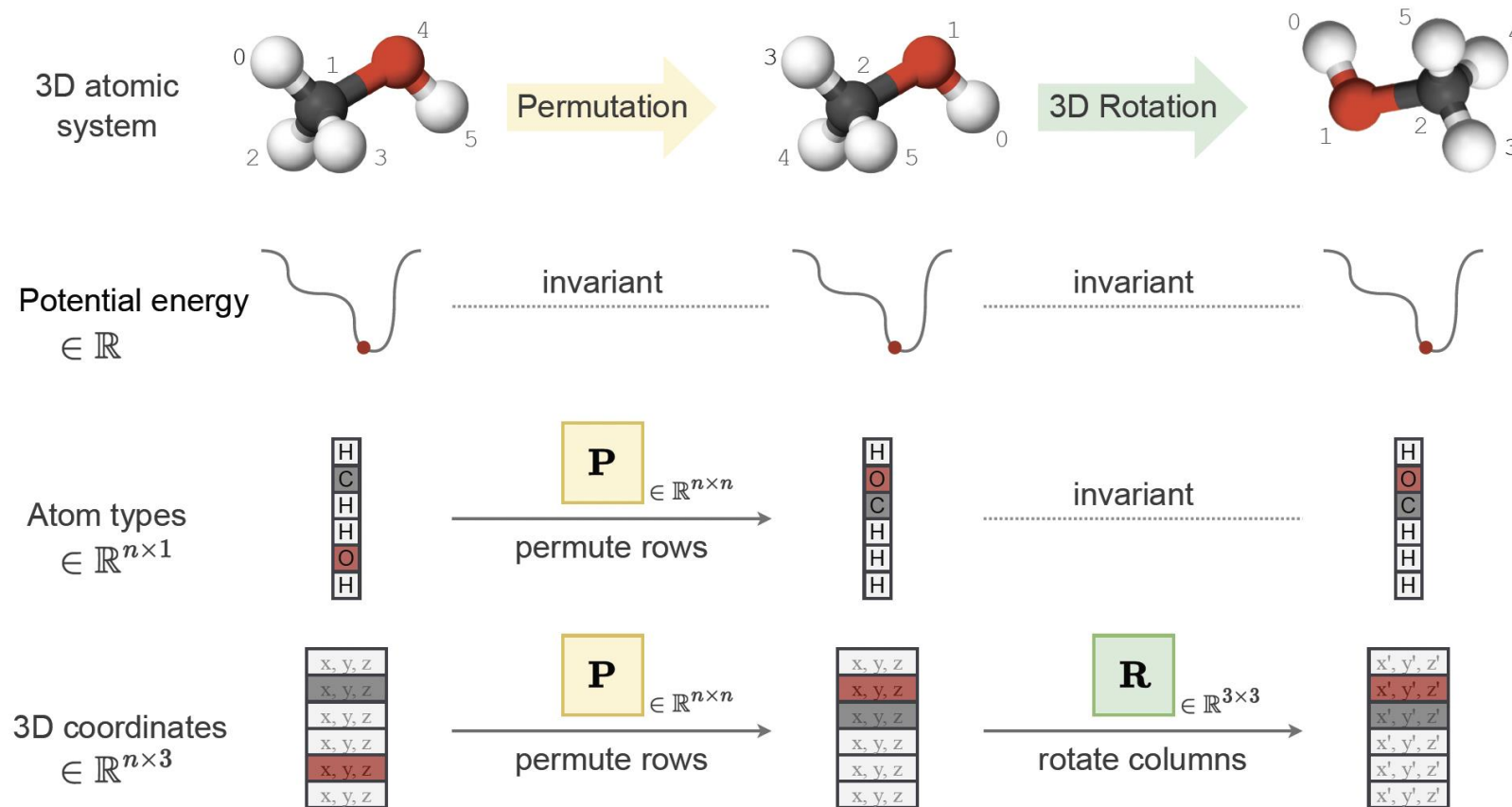
Why do we need geometric graphs?

- We often know geometric attributes (position, velocity, forces) of objects in a physical system (e.g., molecules)
- We would like to include these in a GNN model
- Why not just add these as scalar features?
 - Geometric properties are typically invariant or equivariant under known transformations e.g., translation)
 - Treating them as standard node features may not retain their physical meaning and transformation semantics



Why do we need geometric graphs?

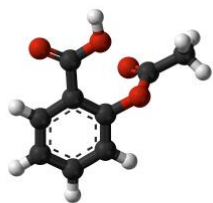
Geometric properties are typically invariant or equivariant under known transformations e.g., translation)



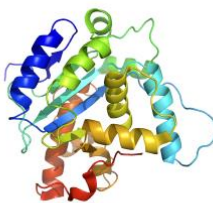


Why do we need geometric graphs?

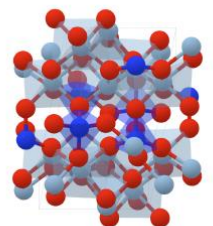
Predicting properties via supervised learning



Small molecules



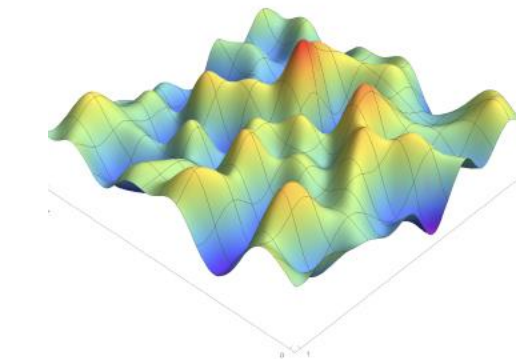
Biomolecules



Materials



Geometric
GNN



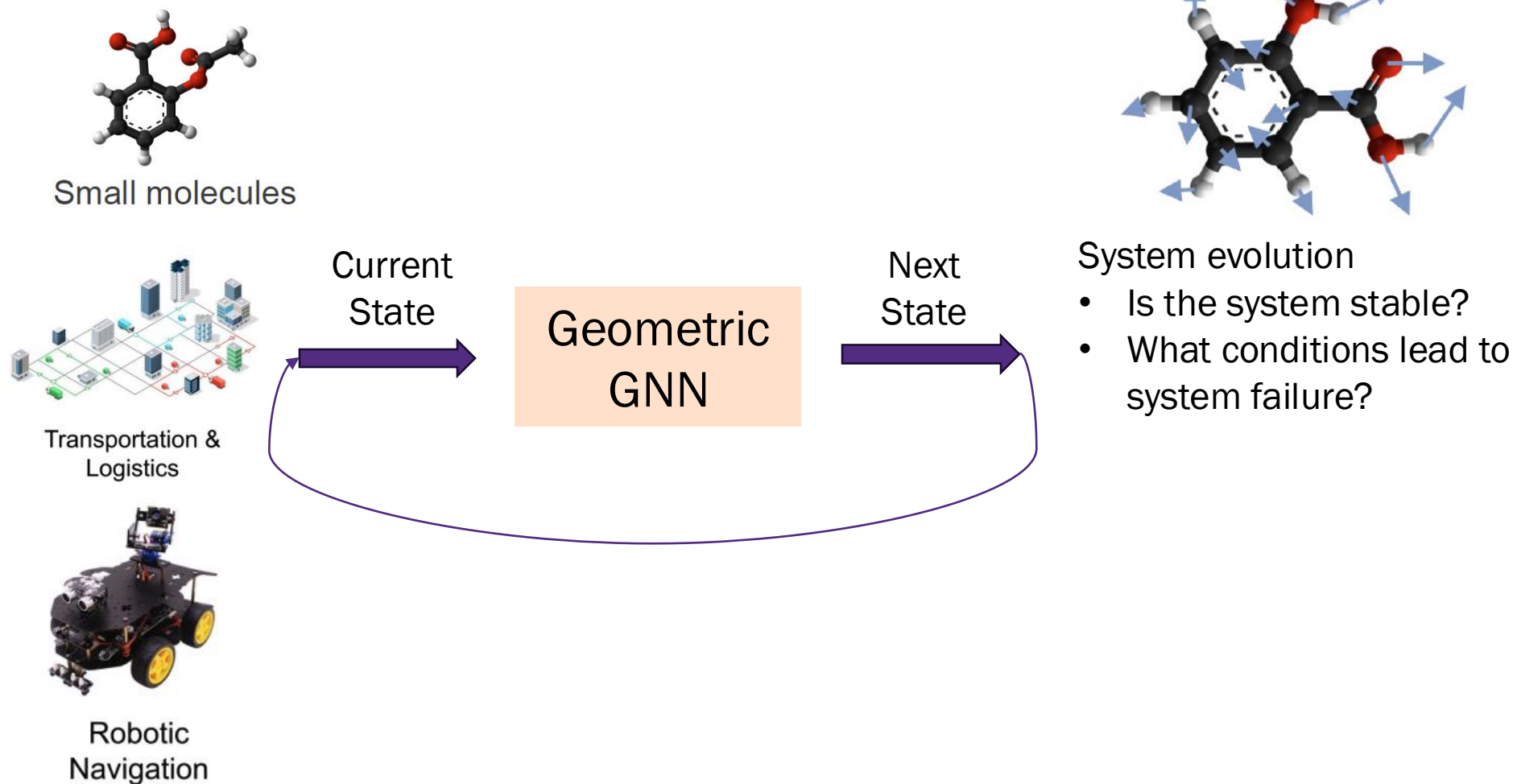
Predictions

- Ligand binding affinity
- Solubility
- Toxicity
- Stability
- ...



Why do we need geometric graphs?

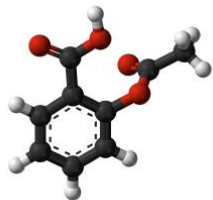
Modeling system dynamics via supervised learning



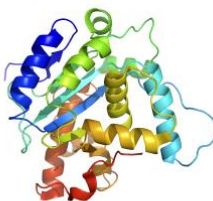


Why do we need geometric graphs?

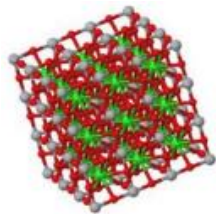
Discovery via **generative modeling**



Small molecules



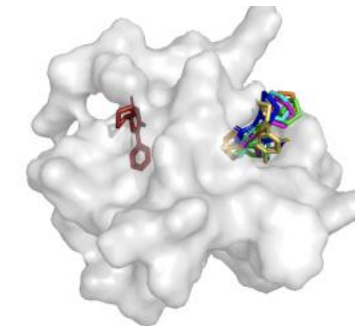
Biomolecules



Inorganic
Crystals



Geometric
GNN



Novel graphs

- Promising drug candidates
- New materials



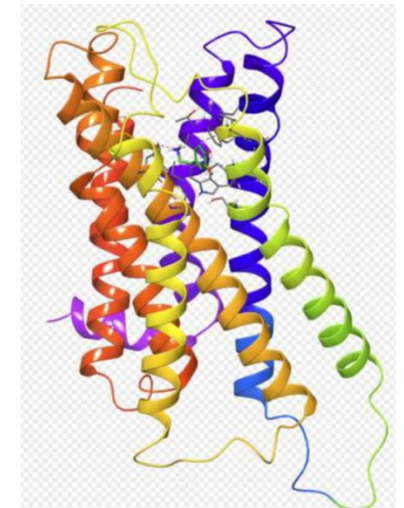
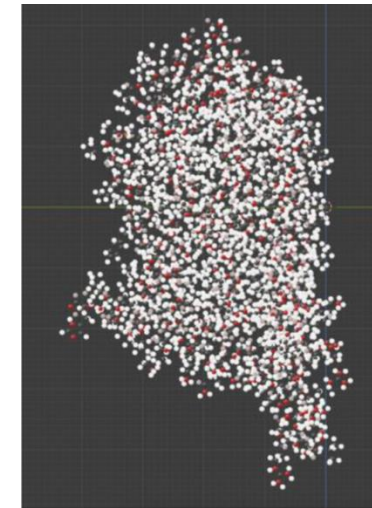
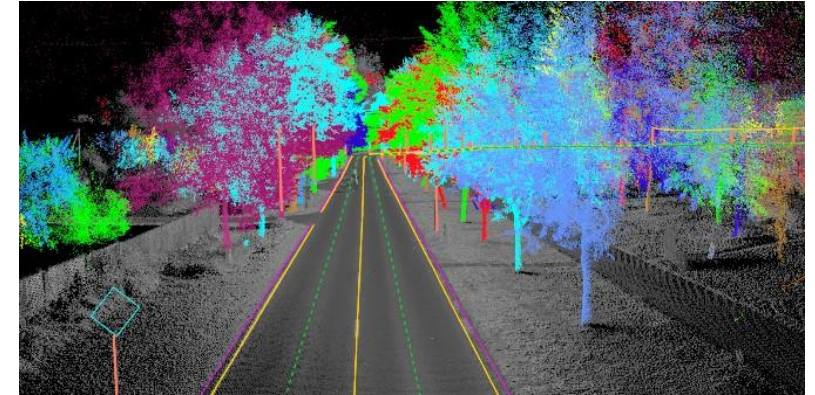
Constructing geometric graphs





Point clouds

- The raw data used to create geometric graphs is usually a 3D point cloud $\{\mathbf{S}, \vec{\mathbf{x}}\}$ where $\mathbf{S}$ represents information at the spatial locations in $\vec{\mathbf{x}}$

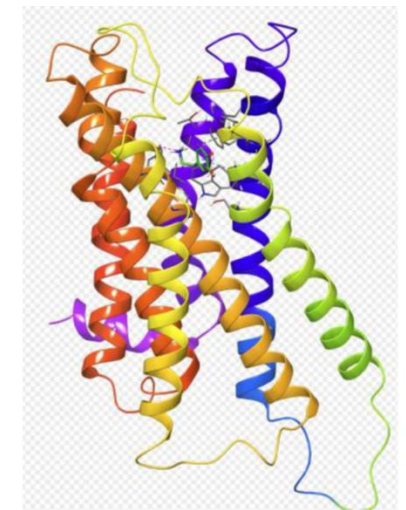
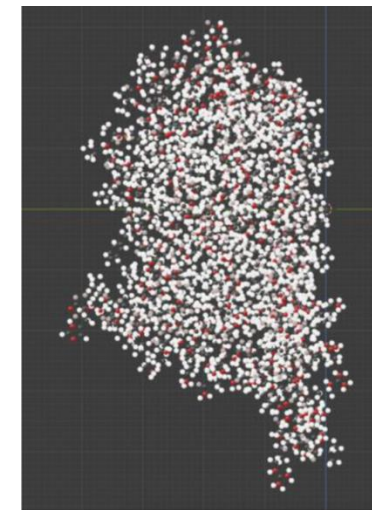
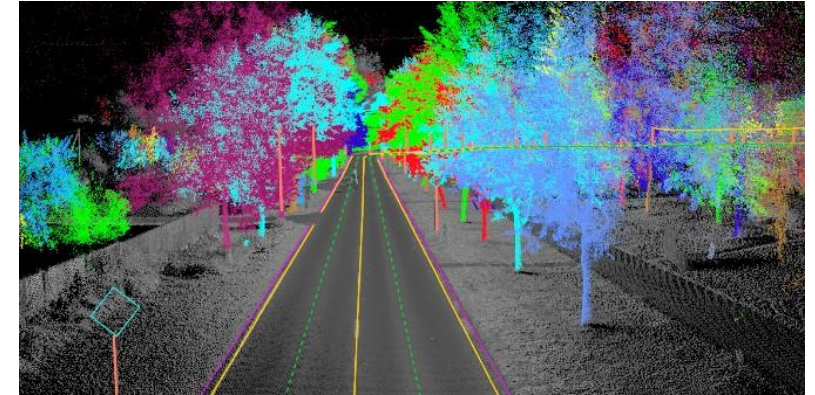




Point clouds

- The raw data used to create geometric graphs is usually a 3D point cloud $\{\mathbf{S}, \vec{\mathbf{x}}\}$ where $\mathbf{S}$ represents information at the spatial locations in $\vec{\mathbf{x}}$
- Can represent a variety of objects / environments
 - Scenes (navigation, virtual reality)
 - Buildings (architecture, engineering)
 - Molecular / atomic systems
 - Human tissues, organs, anatomical structure

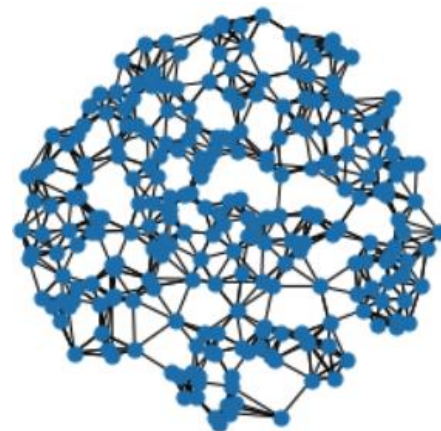
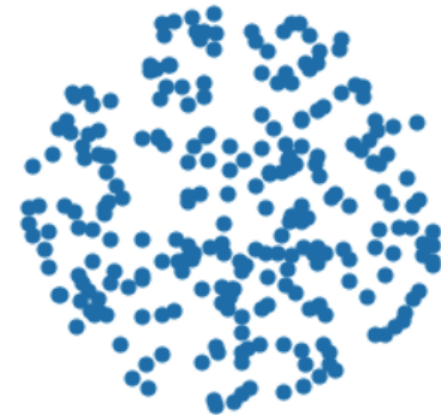
How do we convert
these to graphs?





Point clouds to graphs

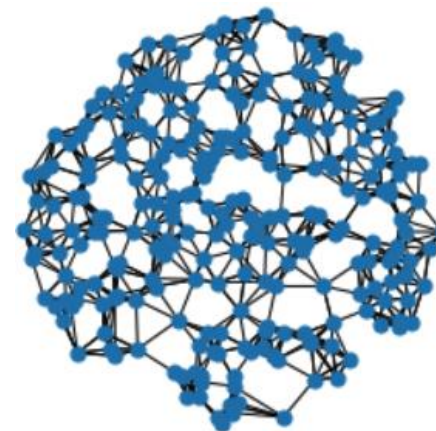
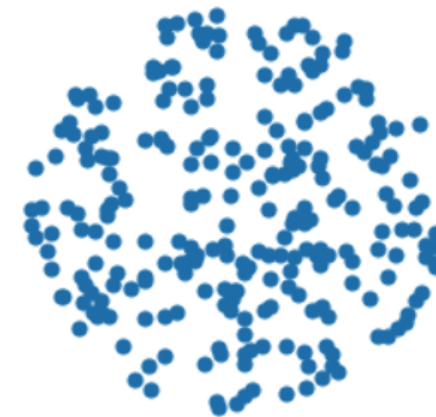
- How should we convert point clouds to graphs? Specifically, which points should be connected via edges?





Point clouds to graphs via a **complete** graph

- **Complete graph** – every point is connected to every other point
- Edges weighted by distance between points
- Most comprehensive representation of the system
- Computationally demanding for larger (more nodes) systems



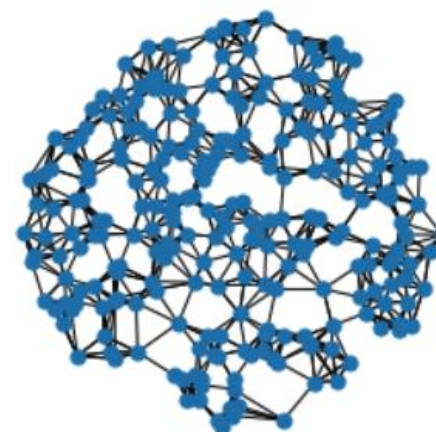
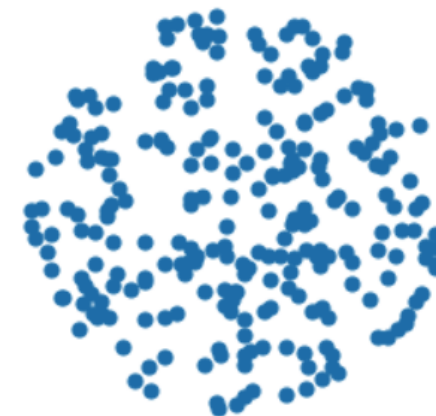


Point clouds to graphs via a **cutoff** graph

- **Cutoff graph** – points i and j connected if their distance $d_{ij} = \|\vec{x}_i - \vec{x}_j\|$ is less than or equal to some cutoff, c .
- Adjacency matrix elements given by

$$a_{ij} = \begin{cases} 1 & \text{if } d_{ij} \leq c \\ 0 & \text{otherwise} \end{cases}$$

- Enforces locality of interactions though multiple GNN layers enable some long range influence
- Can cause issues in molecular simulations if a node moves in/out of range of another node (edge weight is a step function between 0 and 1)

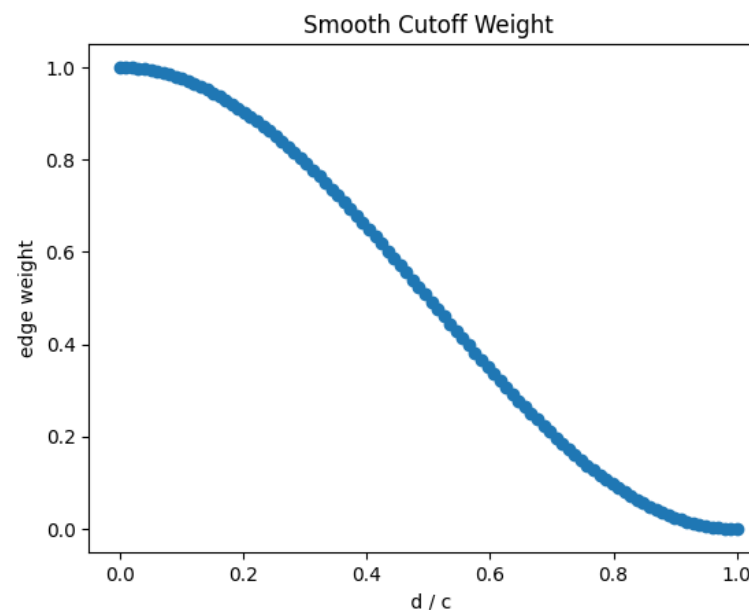
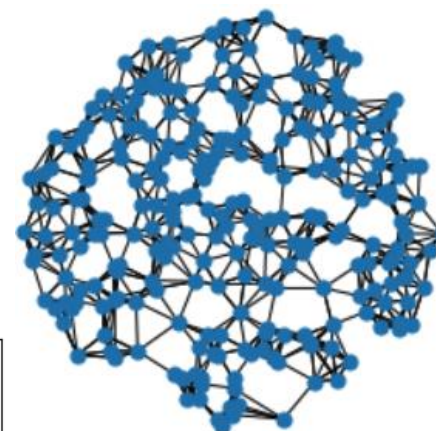
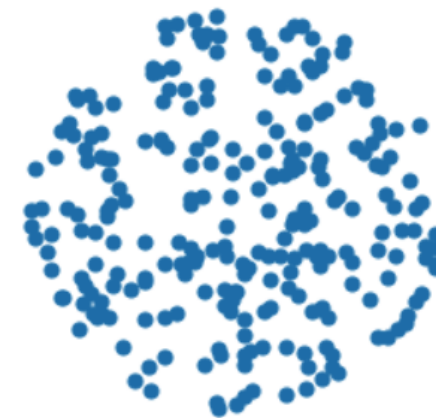




Point clouds to graphs via a **smooth cutoff** graph

- **Smooth cutoff graph** – points i and j connected by a **weighted edge** if their distance $d_{ij} = \|\vec{x}_i - \vec{x}_j\|$ is less than or equal to some cutoff, c .
- Adjacency matrix elements given by

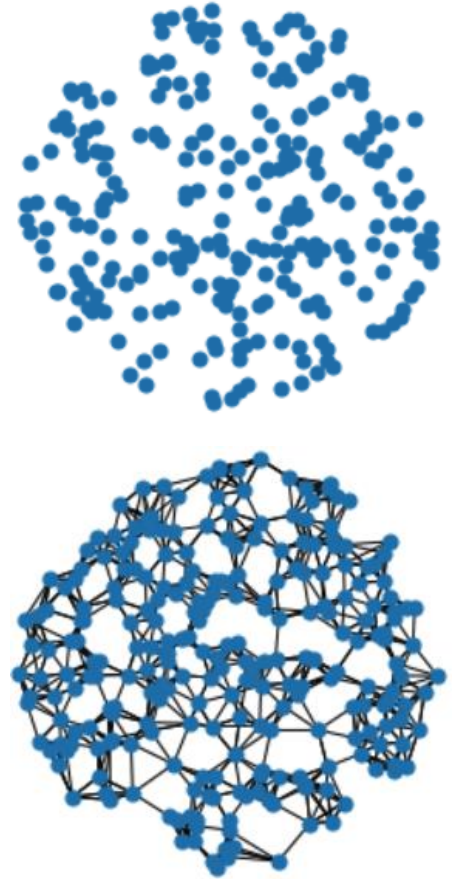
$$a_{ij} = \begin{cases} \frac{1}{2} \left[\cos\left(\frac{\pi d_{ij}}{c}\right) + 1 \right] & \text{if } d_{ij} \leq c \\ 0 & \text{otherwise} \end{cases}$$





Point clouds to graphs

- Which method to use typically requires some domain knowledge
- There are domain specific methods (e.g., in crystalline structures periodic boundary conditions are added)





Point cloud processing with PyTorch Geometric

PyG includes

- Point cloud datasets
- Transformations to convert point clouds to graphs (somewhat limited)

Point Cloud Processing

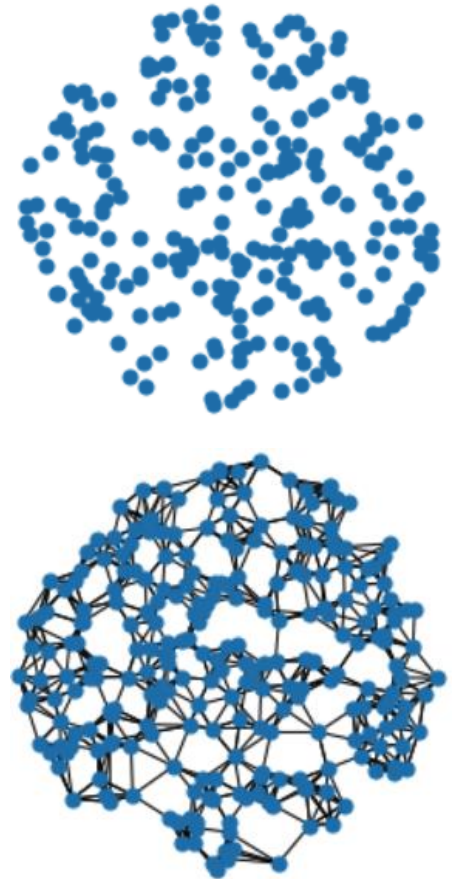
This tutorial explains how to leverage Graph Neural Networks (GNNs) for operating and training on point cloud data. Although point clouds do not come with a graph structure by default, we can utilize  PyG transformations to make them applicable for the full suite of GNNs available in  PyG. The key idea is to create a synthetic graph from point clouds from which we can learn

https://pytorch-geometric.readthedocs.io/en/stable/tutorial/point_cloud.html



Other data pre-processing steps

- Standard data preparation
 - Feature normalization
 - Centering to the origin (subtracting the point cloud centroid)
- Augmentation
 - Euclidean transformations on training samples
 - Corrupted samples (e.g., noise added to position information)





Geometric symmetry





Permutation **equivariance** for node labeling

- Consider a function $f(\mathbf{A}, \mathbf{X}) \rightarrow \mathbb{R}^{|V| \times d_{out}}$ where $\mathbf{A}$ is a graph adjacency matrix and $\mathbf{X}$ is a node feature matrix **containing scalar values**
- Formally, a function $f(\mathbf{A}, \mathbf{X}): \mathbb{R}^{|V| \times |V|} \times \mathbb{R}^{|V| \times d_{in}} \rightarrow \mathbb{R}^{|V| \times d_{out}}$ is **permutation equivariant** if

$$\mathbf{P}f(\mathbf{A}, \mathbf{X}) = f(\mathbf{PAP}^T, \mathbf{PX})$$

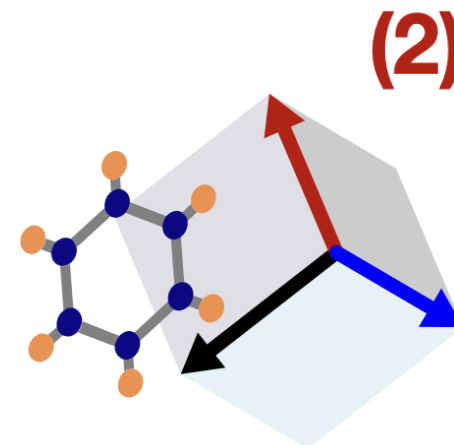
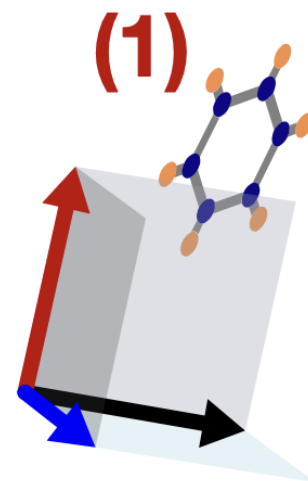
where $\mathbf{P}$ is a permutation matrix

We will need a more
general understanding of
equivariance for geometric
graphs. Let's see why.



Geometric symmetry groups

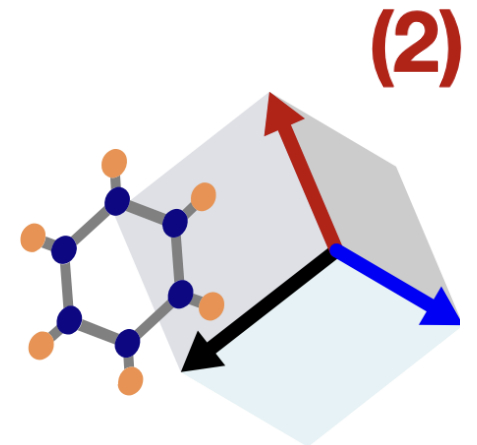
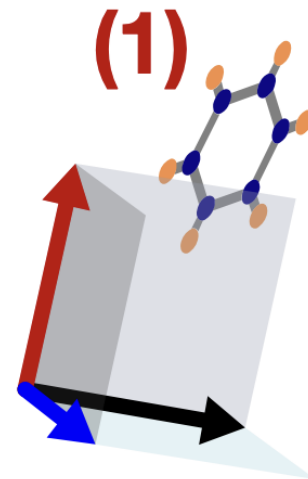
- We want to use geometric graphs to represent objects in a coordinate system
- Different coordinate systems can describe the same geometry
- We can mathematically describe the transformations between coordinate systems with symmetries of Euclidean spaces
 - rotations, reflections, translations





Geometric symmetry of inputs

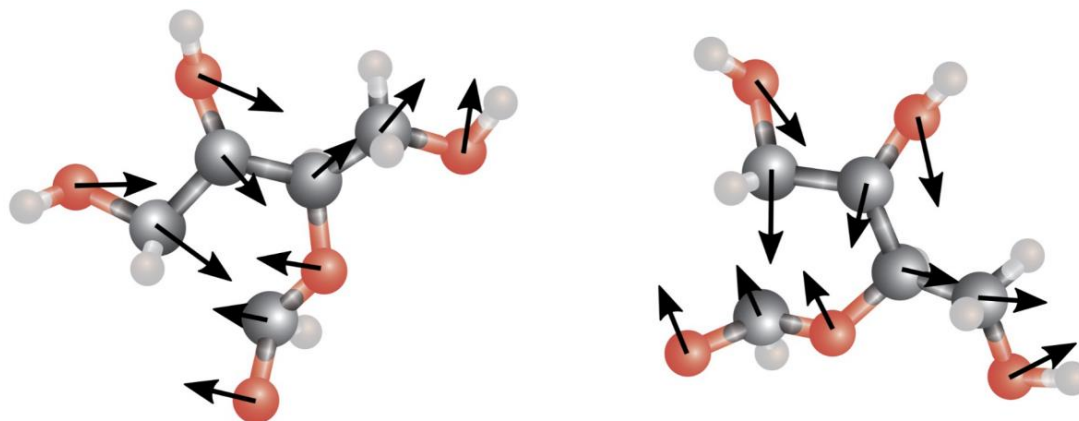
- Different coordinate systems can describe the same geometry
- Standard GNNs will produce unpredictably different answers given tensor inputs (e.g., position vectors) from different coordinate systems
 - This is not just node relabeling, it's a change to the input feature values
- We want to design geometric GNNs that are *symmetry aware*





Geometric symmetry of outputs

- In standard GNNs, we need to ensure the **encoder** is equivariant to node relabeling
 - Node embedding updates are invariant
 - Graph embedding matrix (i.e., matrix of node embeddings) is equivariant
- For a geometric graph that **outputs tensors** (e.g., force prediction), **the decoder also needs to be equivariant** to coordinate system changes
- Example: predicted molecular forces should be equivariant to rotations





Groups

A **group** is a collection of reversible operations that act on elements of spaces.



Groups

A **group** is a collection of reversible operations that act on elements of spaces.

An operation in a group is a **concept** that may have different **realizations** (implementations) in different spaces



Groups

A **group** is a collection of reversible operations that act on elements of spaces.

An operation in a group is a **concept** that may have different **realizations** (implementations) in different spaces

Consider the **concept** of swapping nodes 1 & 2 in a graph



Groups

A **group** is a collection of reversible operations that act on elements of spaces.

An operation in a group is a **concept** that may have different **realizations** (implementations) in different spaces

Consider the **concept** of swapping nodes 1 & 2 in a graph

1. Let X be the space of all adjacency matrices. Swapping nodes 1 and 2 requires swapping rows 1 and 2 of the adjacency matrix.



Groups

A **group** is a collection of reversible operations that act on elements of spaces.

An operation in a group is a **concept** that may have different **realizations** (implementations) in different spaces

Consider the **concept** of swapping nodes 1 & 2 in a graph

1. Let X be the space of all adjacency matrices. Swapping nodes 1 and 2 requires swapping rows 1 and 2 of the adjacency matrix.
2. Let Y be the space of all adjacency lists. Swapping nodes 1 and 2 requires setting 0 to 1 and 1 to 0 in all edges in the list.

Same operation concept, different realizations.



Groups

A **group** is a collection of reversible operations that operate on elements on spaces.

The operations in a group can be composed and obey four rules:

1. **Closure:** If operations A and B are in the same group, then the **composite operation** A followed by B is also in the group
2. **Associativity:** If operations A , B , and C are in the same group then $(A \circ B) \circ C = A \circ (B \circ C)$ where $\circ$ indicates composition
3. **Identity:** For every group, there exists an operation I such that $Ix = x$ for every element $x \in X$
4. **Inverse:** Every operation is reversable, $A \circ A^{-1} = I$



Group example

Rotations of a square

- Concepts: Rotate r degrees where $r \in \{0, 90, 180, 270\}$
- **Closure:** Compose any two rotations results in a rotation in the set (assuming we consider a rotation of >360 to be the modulo of 360)
- **Associativity:** Rotating by any two values in the set, and then by a third is independent of order (assuming the modulo is done after all rotations)
- **Identity:** rotation by 0
- **Inverse:** Each has an inverse $90 \leftrightarrow 270$, $180 \leftrightarrow 180$



Equivariance on groups

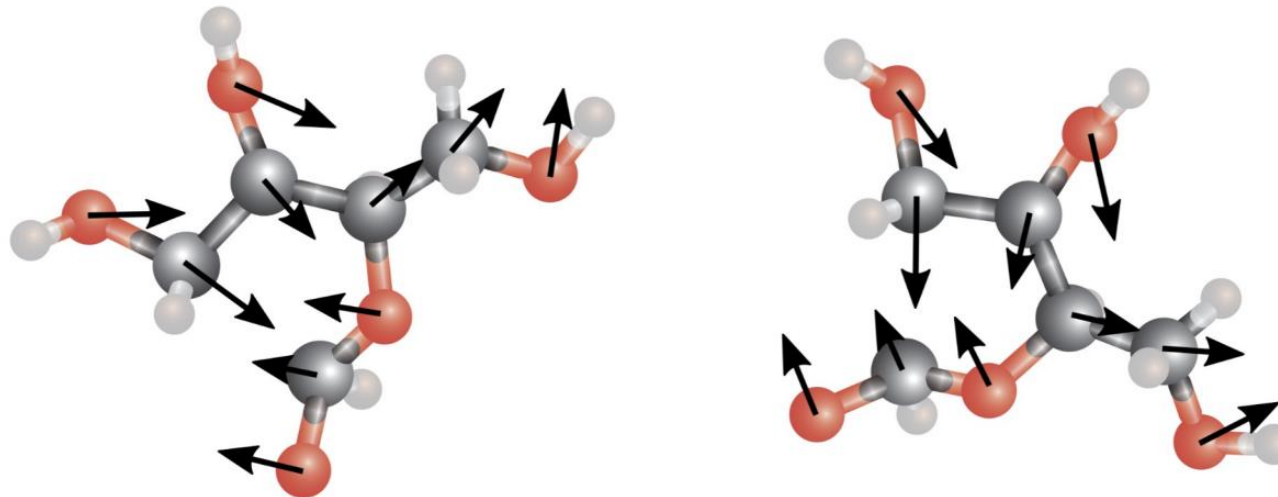
A function $f: X \rightarrow Y$ is equivariant to the group P if for any group element, $\rho \in P$, f satisfies

$$f(\rho_X \circ x) = \rho_Y \circ f(x)$$

where we note ρ_X and ρ_Y represent the instantiations of the group concept ρ in the spaces X and Y , respectively

- Example: ρ_X and ρ_Y represent the rotation transformation

Technical note, ρ_X and ρ_Y are related. Specifically, for their must exist a homomorphism, ψ , such that $\psi(\rho_X) = \rho_Y$

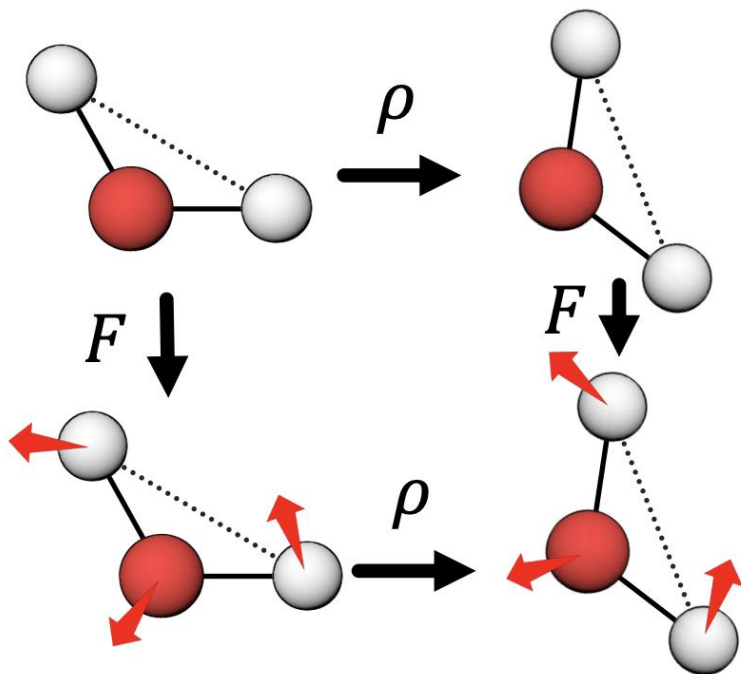




3D rotation equivariance

$$F(\rho \circ x) = \rho \circ F(x)$$

The equation implies that applying the rotation ρ on the input has the same effect as applying it to the output



Rotating the molecule (**input**) and then calculating the forces, F , is the same as calculating the forces (**output**) and then rotating them.



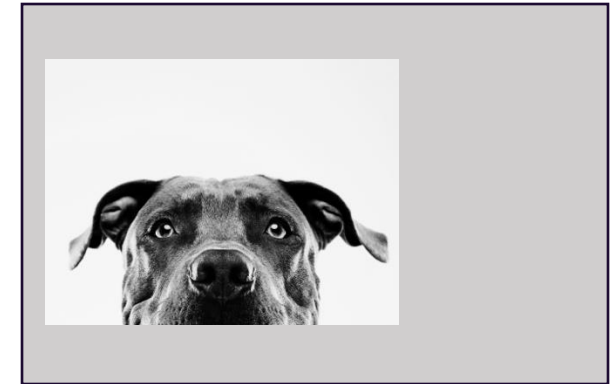
Invariance

- A function $f: X \rightarrow Y$ is invariant to the group P if for any group element, $\rho \in P$, f satisfies

$$f(\rho_X \circ x) = f(x)$$

where we note ρ_X represent the instantiation of the concept ρ in the space X

- In this extended group formulation, *invariance* is a special case of *equivariance*



Output label: dog

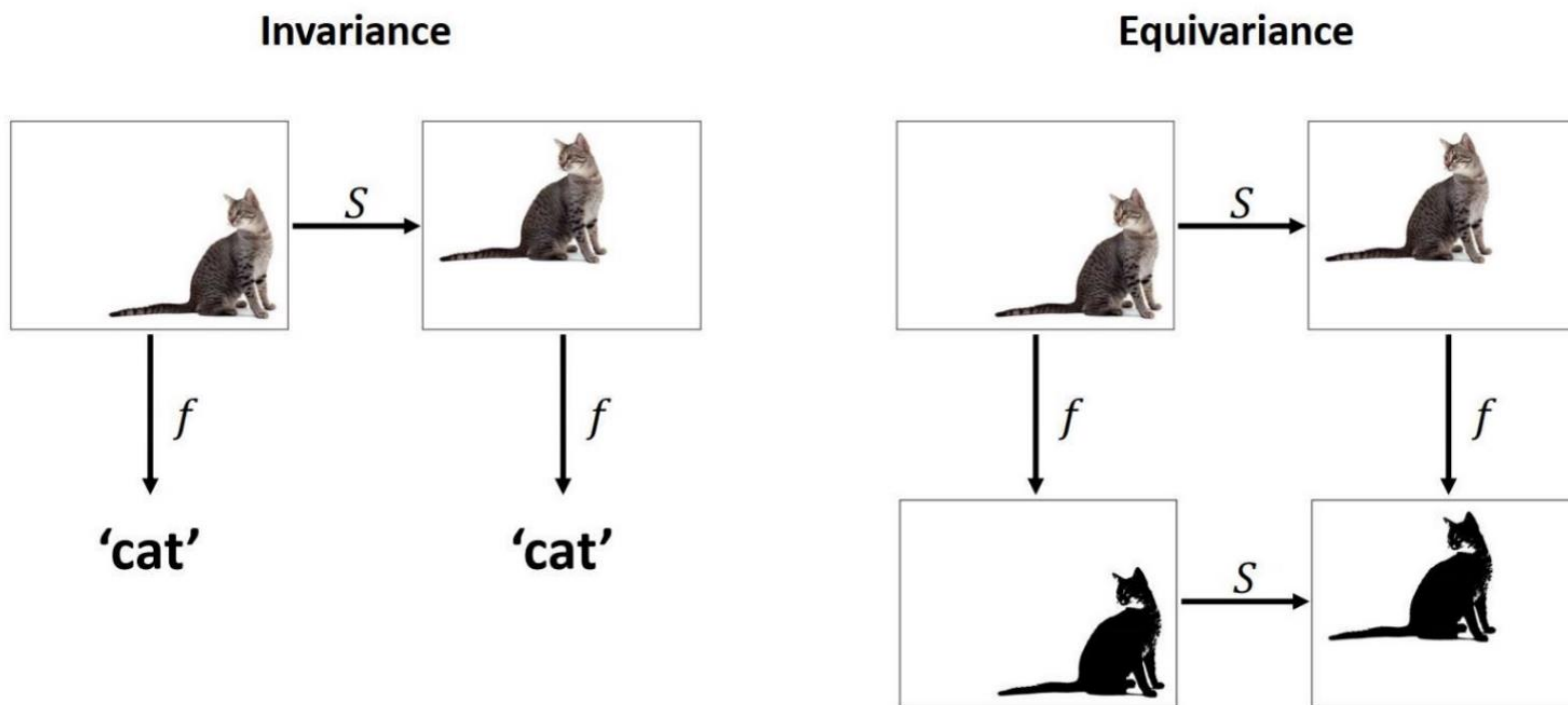
After rotation,
translation



Stil a dog

Invariance & equivariance on images

- Consider rotations, reflections and translations in the image domain
 - Classification: label output (**scalar**) should be **invariant**
 - Segmentation: pixel coordinates (**tensor**) should be **equivariant**





Invariance and equivariance in Euclidean space

- For geometric graphs, we want equivariance on
 - $SO(3)$: the group of all rotations in 3D space that are distance preserving
 - $OD(3)$: the group of rotations and reflections in 3D space that are distance preserving
 - $TD(3)$: the group of translations that act via vector addition



Invariance and equivariance in Euclidean space

- For geometric graphs, we want equivariance on
 - $SO(3)$: the group of all rotations in 3D space that are distance preserving
 - $OD(3)$: the group of rotations and reflections in 3D space that are distance preserving
 - $TD(3)$: the group of translations that act via vector addition
- In practice, this means we require
 - **Structured outputs (tensors)** such as forces to be **rotation equivariant** and translation invariant
 - **Scalar outputs** such as energy to be **rotation and translation invariant**



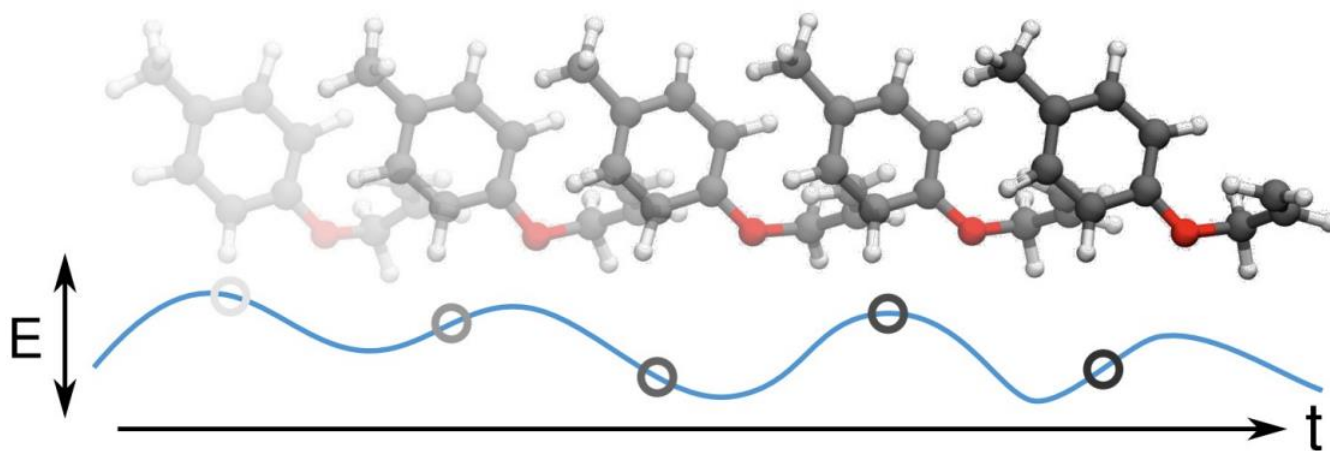
Molecular dynamics as a templar



Molecular dynamics simulations

Standard methods of simulating the stable geometric structure of molecules require computationally expensive quantum mechanical calculations

- Energy E (scalar)
- Forces F (tensor)

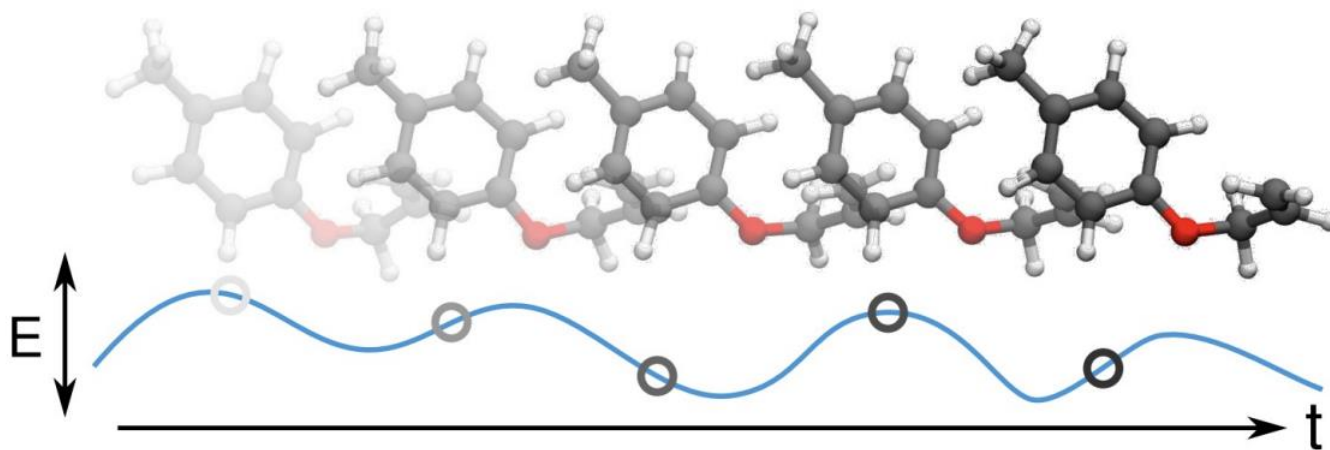


$$\hat{H}\Psi = E\Psi$$

Method	Complexity
Hartree Fock	$O(n^3) - O(n^4)$
Density Functional Theory	$O(n^3) - O(n^4)$
MP2	$O(n^5)$
CCSD	$O(n^6)$
CCSD(T)	$O(n^7)$
Full CI	$O(n!)$

Molecular dynamics simulations

- Can be used for various applications
 - Protein folding prediction
 - Materials property prediction
 - Self-assembly of nanostructures

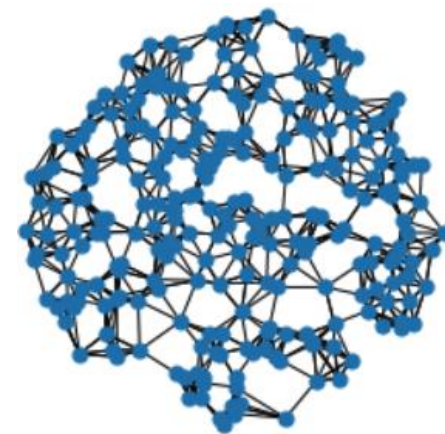
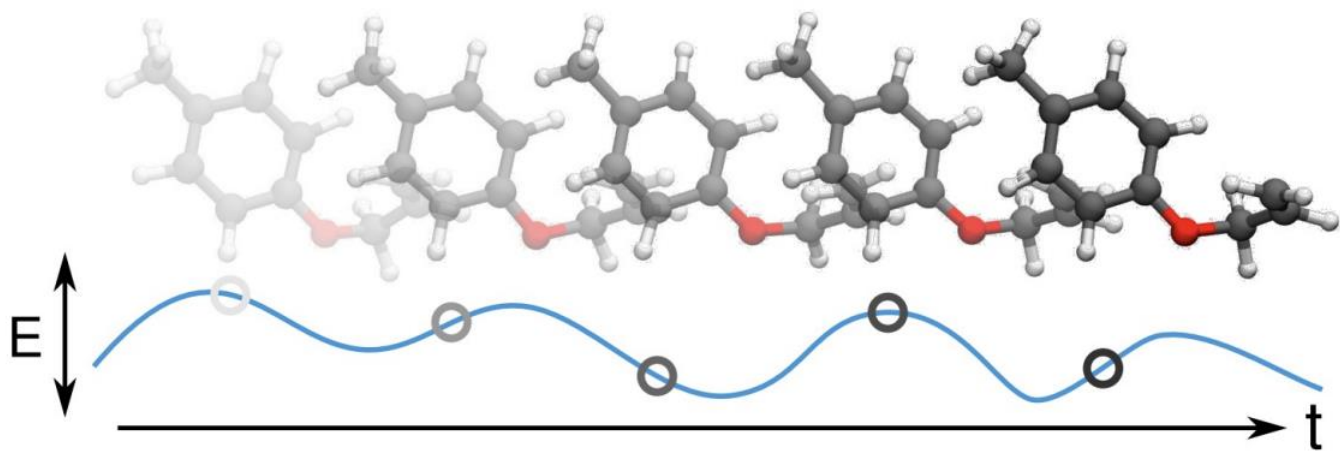


$$\hat{H}\Psi = E\Psi$$

Method	Complexity
Hartree Fock	$O(n^3) - O(n^4)$
Density Functional Theory	$O(n^3) - O(n^4)$
MP2	$O(n^5)$
CCSD	$O(n^6)$
CCSD(T)	$O(n^7)$
Full CI	$O(n!)$

Molecular dynamics simulations

- Geometric graphs to reduce computational costs & increase accuracy?
- Inputs: molecular graphs with scalar atom types and tensor positions
- Outputs:
 - Molecular total energy (scalar, invariant)
 - Forces acting on each atom (tensor, equivariant)





Invariant geometric GNNs





Invariant geometric GNNs

- Key idea of **invariant** geometric GNNs is to **convert geometric input tensors to** informative **scalar quantities** such as distances and angles
- The input scalar quantities are invariant to Euclidean transformations
- Intermediate node representations and output predictions are guaranteed to be invariant
- Limitation – all such approaches rely on a set of precomputed local invariants (**feature engineering**)



Invariant geometric GNNs

- As with standard GNNs, goal is to learn node embeddings that can be used by a prediction head (decoder) for inference
- In general, the **scalar** embedding update for node i at layer t formed via message passing

$$\mathbf{s}_i^{(t+1)} \equiv \text{UPD} \left(\mathbf{s}_i^{(t)}, \bigoplus_{j \in N_i} \text{MSG}(\mathbf{S}^t, \vec{\mathbf{x}}, i, j) \right)$$

learnable functions aggregation



Distance-based invariant geometric GNNs

- Scalarize node (atom) coordinates by precomputing the pairwise distance between every pair of nodes u_i, u_j

$$d_{ij} = \|\vec{x}_{ij}\| = \sqrt{(\vec{x}_i - \vec{x}_j) \cdot (\vec{x}_i - \vec{x}_j)}$$

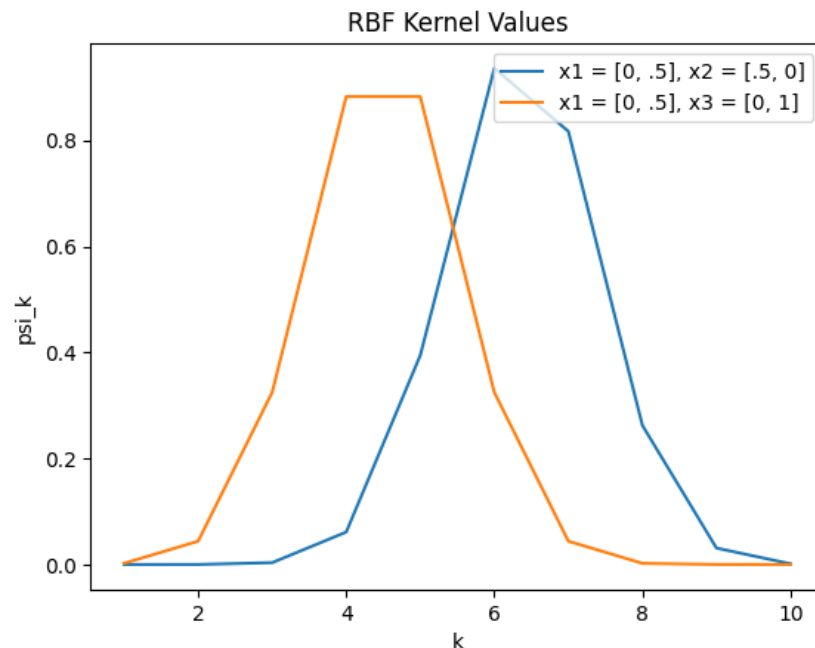
- Distances are invariant to rotations, reflections and translations
- The **scalar** embedding update for node i at layer t is now

$$\mathbf{s}_i^{(t+1)} \equiv \text{UPD} \left(\mathbf{s}_i^{(t)}, \bigoplus_{j \in N_i} \text{MSG}(\mathbf{S}^t, d_{ij}, i, j) \right)$$



SchNet distance-based invariant GNN

- SchNet precomputes the scalar distances between each node (atom)
- For each node pair, the **distances are used to form a feature vector**, $\Psi(d_{ij}) = [\psi_1, \dots, \psi_a]$ using radial basis functions (RBF) where
 - a is the embedding dimension (tuning parameter)
 - $\psi_k = e^{-(d_{ij}-u_k)^2/(2\sigma^2)}$, $u_k = k/(a-1)$, $\sigma = 1/(a-1)$



RBF distance representation converts scalar distance between two points to a vector of distances from fixed coordinates, u_k . Enables a linear model to have non-linear response to different distance regimes.

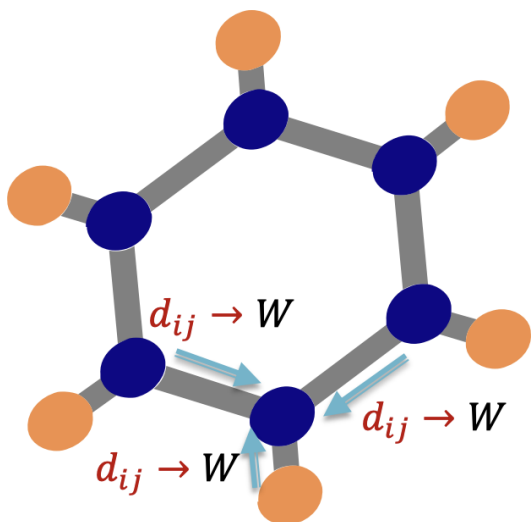


SchNet distance-based invariant GNN

- SchNet node scalar embedding update for node i at layer t is now given by

$$\mathbf{s}_i^{(t+1)} = \mathbf{s}_i^{(t)} + \sum_{j \in N_i} \mathbf{s}_j^{(t)} \odot \mathbf{W}^{(t)} \psi(d_{ij})$$

learnable
matrix



- Weight matrix $\mathbf{W}^{(t)}$ acts on relative distances between neighbor nodes
- $\mathbf{W}^{(t)}$ controls the interaction of scalar features in $\mathbf{s}_i^{(t)}$ and $\mathbf{s}_j^{(t)}$
- Atom interaction is distance dependent



DimeNet distance and angle-based invariant GNNs

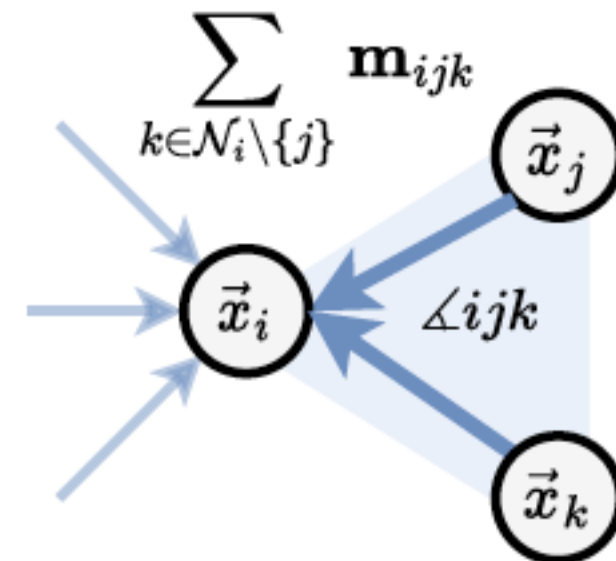
- Total molecular potential energy is the sum of four parts

$$E = E_{bonds} + E_{angle} + E_{torsion} + E_{non-bonded}$$

- SchNet only considers atom types and pairwise distances

- DimeNet**

- Considers bond angles and distance between node (atom) triplets (i, j, k)
- As with SchNet, uses basis functions to form distance and angle features
 - Distances between each (i, j) and (i, k) represented by Spherical Bessel Functions (SBF)
 - Angles formed by the triplet represented by Spherical Harmonics





Limitation of invariant geometric GNNs

- All invariant geometric GNNs rely on a set of precomputed local invariants
- In other words, the raw geometric relations are converted to informative values
- This is **feature engineering**
- Can a model that acts directly on the geometric tensors be more expressive?



Equivariant geometric GNNs





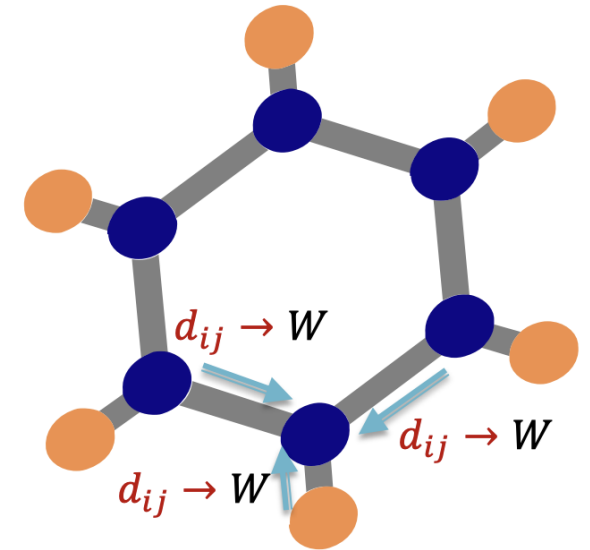
PaiNN Equivariant GNN

- Includes scalar and tensor (vector) features and messages
- Scalar messages are formed by applying a learnable weight matrix $\mathbf{W}$ to relative distances represented by RBF (as in SchNet)

$$\mathbf{m}_i^{(t)} = \mathbf{s}_i^{(t)} + \sum_{j \in N_i} \mathbf{s}_j^{(t)} \odot \mathbf{w}_1^{(t)} \psi(d_{ij})$$

- Tensor messages are formed by scaling tensor features by weights determined by the relative distances and input scalar features

$$\vec{\mathbf{m}}_i^{(t)} = \vec{\mathbf{v}}_i^{(t)} + \sum_{j \in N_i} \mathbf{s}_j^{(t)} \odot \mathbf{w}_2^{(t)} \psi(d_{ij}) \odot \vec{\mathbf{v}}_j^{(t)} + \sum_{j \in N_i} \mathbf{s}_j^{(t)} \odot \mathbf{w}_3^{(t)} \psi(d_{ij}) \odot \vec{\mathbf{x}}_{ij}^{(t)}$$



summation over vector scaling is
equivariant to input changes



PaiNN Equivariant GNN

- Node embedding updates performed via residual connections
- Node scalar embedding updates

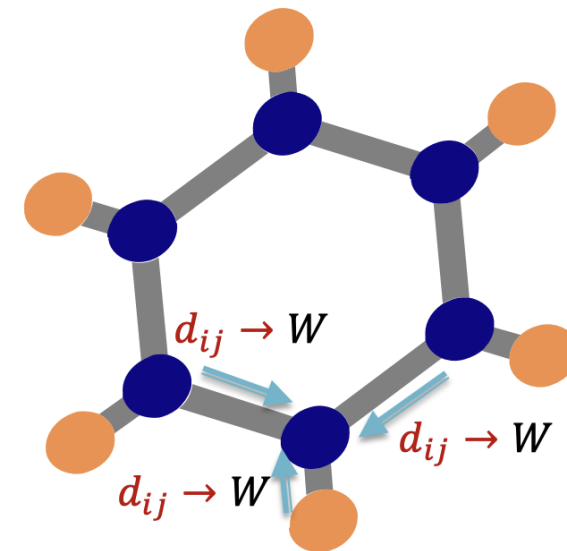
$$\Delta \mathbf{s}_i^{(t)} = \mathbf{m}_i^{(t)} = \mathbf{s}_i^{(t)} + \sum_{j \in N_i} \mathbf{s}_j^{(t)} \odot \mathbf{w}_1^{(t)} \psi(d_{ij})$$

$$\mathbf{s}_i^{(t+1)} = \mathbf{s}_i^{(t)} + \Delta \mathbf{s}_i^{(t)}$$

- Node tensor embedding updates

$$\Delta \vec{\mathbf{v}}_i^{(t)} = \vec{\mathbf{m}}_i^{(t)} = \vec{\mathbf{v}}_i^{(t)} + \sum_{j \in N_i} \mathbf{s}_j^{(t)} \odot \mathbf{w}_2^{(t)} \psi(d_{ij}) \odot \vec{\mathbf{v}}_j^{(t)} + \sum_{j \in N_i} \mathbf{s}_j^{(t)} \odot \mathbf{w}_3^{(t)} \psi(d_{ij}) \odot \vec{\mathbf{x}}_{ij}^{(t)}$$

$$\vec{\mathbf{v}}_i^{(t+1)} = \vec{\mathbf{v}}_i^{(t)} + \Delta \vec{\mathbf{v}}_i^{(t)}$$





PaiNN Equivariant GNN

- By stacking multiple PaiNN layers
 - vector features after final layer are tensors equivariant to input coordinate changes
 - vector features can be used directly for force (tensor) prediction

